

Using Cascade Models to Reduce Inference Time and Resource Usage in Edge Computing Scenarios

Houda Mesly and Yelyzaveta Ploskonos
Master 2 UBINET
Université Côte d’Azur

December 21, 2025

Introduction

In the age of ubiquitous edge AI, where models run on everything from sensors to smartphones, traditional inference methods fall short. In such environments, achieving low inference latency while maintaining high accuracy is a major challenge. These limitations highlight the need for alternative strategies that balance speed, accuracy and resource efficiency. In this project, we investigate cascade models for a regression task: predicting an individual’s income from demographic and work-related features. We use the Income dataset introduced by Ding et al. (2024) built from the American Community Survey (ACS), and we design and evaluate cascades that combine:

- a small, fast model (linear regression or small MLP) with a confidence estimate. - a large, accurate MLP with a probabilistic output.

The main objective is to use probabilistic confidence estimation (Gaussian NLL heteroscedastic models) from the first model to decide whether the obtained prediction is reliable or we should invoke the second model’s prediction. This mechanism optimizes the accuracy, efficiency and trade-off by reserving computational resources only for ambiguous cases. Our implementation demonstrates significant reductions in inference time and usage while maintaining accuracy in terms of predictions. Which highlights the effectiveness cascade methodologies in edge computing environments where input data exhibits variable complexity and enabling resource allocation based on prediction uncertainty.

The contributions of this work are as follows:

- We analyse and adapt cascade architectures for regression tasks in edge computing environments.
- We implement uncertainty-aware small and large models and study their behaviour on real census data.
- We evaluate the effect of model size, inference time, confidence calibration, and loss functions on cascade performance.
- We quantify the resource savings achieved by cascades in terms of computation, latency, and bandwidth usage.

Overall, our results demonstrate that cascade models provide a practical and effective mechanism for reducing resource consumption while maintaining competitive predictive performance in edge computing scenarios.

1 Question 1: Cascade Models for Edge Computing

Cascade models were originally introduced by Viola and Jones [1] as a sequential architecture composed of multiple classifiers of increasing complexity. At each stage, the model either produces a prediction or forwards the input to the next stage if its confidence is insufficient. This mechanism yields substantial computational savings because the majority of inputs are handled by early, inexpensive stages, while only ambiguous or difficult cases progress deeper into the cascade.

Following this original design principle, our cascade also relies on early rejection: the lightweight model rapidly processes most inputs, and only the uncertain ones are escalated to a more complex and computationally expensive model. This behavior makes cascades naturally aligned with the constraints of edge computing.

Edge devices often operate under strict limitations on latency, energy consumption, bandwidth, and memory capacity. In many real-world applications, not all inputs require high-cost computation. Cascade models provide a principled way to adapt the computational effort to the difficulty of each input, enabling efficient on-device inference while preserving accuracy when needed. By allowing easy cases to be resolved locally and forwarding only harder cases to more powerful models—either on the same device or a nearby MEC server—cascades effectively balance accuracy with resource usage.

1.1 Scenarios Where Cascades Improve Edge Computing

Real-time mobile analytics. Applications such as activity recognition or mobile income prediction can process most “easy” cases using a lightweight on-device model. Only ambiguous cases are forwarded to a remote server running a larger model, reducing latency and preserving battery life.

Video analytics at the edge (smart cameras). Embedded cameras can run a fast model to immediately discard non-relevant frames (e.g., no person or vehicle present). Only potentially important frames are sent to a heavier model on an MEC server, substantially reducing computation and network traffic.

IoT sensor networks. IoT devices often generate large volumes of data. A small local model can filter or predict values with confidence, transmitting only uncertain or anomalous measurements to the cloud for further analysis, and thereby reducing bandwidth usage.

Autonomous and safety-critical systems. In robotics or autonomous drones, delayed decisions can be costly. Cascades enable rapid responses for easy cases using the fast model, while more challenging inputs are handed off to a slower but more accurate model, ensuring both efficiency and safety.

1.2 Advantages for Edge Computing

- **Reduced inference time:** Most inputs are processed by the fast model, significantly lowering average latency.
- **Lower computational and energy usage:** Lightweight stages reduce CPU/GPU load and extend the battery life of the device.
- **Reduced bandwidth:** Only low-confidence cases are transmitted to remote servers or MEC nodes.

- **Scalable deployment:** Cascades adapt naturally to heterogeneous edge environments with varying computational capacities.

In summary, cascade models provide a flexible and efficient framework for designing intelligent edge systems that achieve a favorable trade-off between accuracy, latency, and resource consumption.

2 Question 2: Assessing Confidence in Regression Tasks

In a cascade system, the fast model must provide not only a prediction but also an estimate of its *confidence*. For regression tasks, such confidence can be obtained through *probabilistic regression*, where the model predicts both a mean value and an input-dependent variance. This enables the system to quantify uncertainty and determine whether a prediction can be trusted or should instead be escalated to the more accurate (but more expensive) model in the cascade.

In this work, we employ the Gaussian Negative Log-Likelihood (NLL) loss to learn *heteroscedastic* uncertainty-uncertainty that varies across inputs.

2.1 Gaussian Negative Log-Likelihood for Heteroscedastic Regression

Given an input x and target variable y , a heteroscedastic regression model outputs:

$$\mu(x), \quad \log \sigma^2(x),$$

where $\mu(x)$ is the predictive mean and $\sigma^2(x)$ is the input-dependent variance. Assuming a Gaussian likelihood,

$$y \sim \mathcal{N}(\mu(x), \sigma^2(x)),$$

the loss for a single example becomes:

$$\mathcal{L}_{\text{NLL}}(x, y) = \frac{1}{2} \left[\log \sigma^2(x) + \frac{(y - \mu(x))^2}{\sigma^2(x)} \right].$$

Minimizing this loss encourages the model to:

- predict *small variance* when the target is predictable from the input,
- predict *large variance* when the data are noisy or ambiguous.

As such, Gaussian NLL provides a principled confidence estimator for cascade-based routing.

2.2 Synthetic Example with Non-Uniform Noise

To illustrate the behavior of heteroscedastic regression, we construct a simple 1D dataset:

$$y = \sin(x) + \epsilon(x), \quad x \in [-3, 3],$$

where the noise level depends on the input:

$$\epsilon(x) \sim \begin{cases} \mathcal{N}(0, 0.1^2), & x < 0, \\ \mathcal{N}(0, 0.5^2), & x \geq 0. \end{cases}$$

Figure 1 visualises the resulting data together with the underlying $\sin(x)$ function.

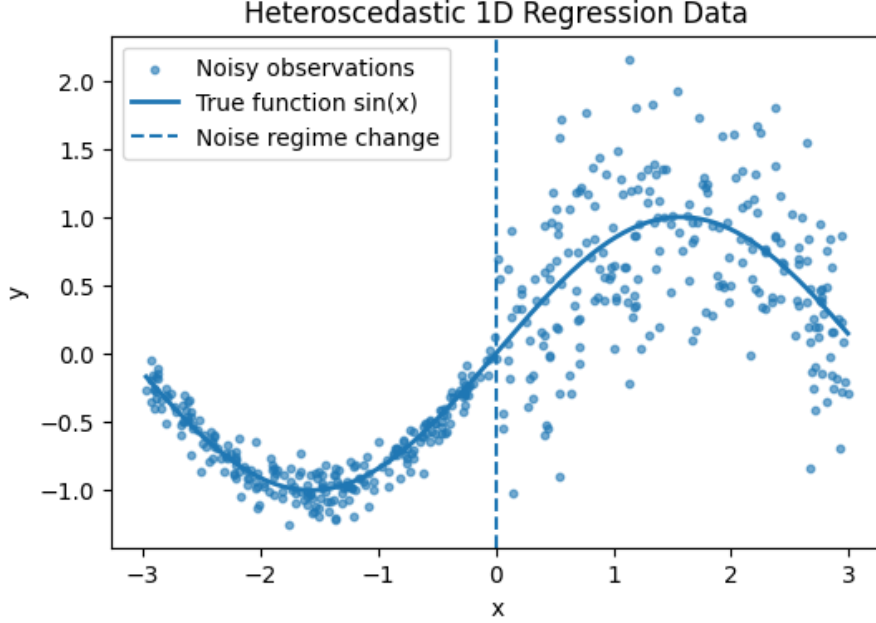


Figure 1: Heteroscedastic 1D regression dataset. Noise is low for $x < 0$ and high for $x > 0$. The model must learn both the mean function and the spatially varying uncertainty.

2.3 Neural Network Trained with Gaussian NLL

We train a neural network with two output heads:

- a mean head $\mu(x)$,
- a log-variance head $\log \sigma^2(x)$.

The model successfully captures:

1. the underlying function $\sin(x)$,
2. low uncertainty in the low-noise region ($x < 0$),
3. high uncertainty in the high-noise region ($x > 0$).

The predicted mean and uncertainty band ($\mu \pm 2\sigma$) are shown in Figure 2.

2.4 Interpreting Uncertainty Estimates

Table 1 reports the uncertainty predicted at the representative test points.

x	Predicted mean $\mu(x)$	Predicted $\sigma(x)$
-2.0	-0.918	0.106
-1.0	-0.860	0.104
0.0	0.054	0.253
1.0	0.809	0.502
2.0	0.940	0.472

Table 1: Predicted uncertainty increases in regions with higher observational noise, confirming the reliability of Gaussian NLL as a confidence estimator.

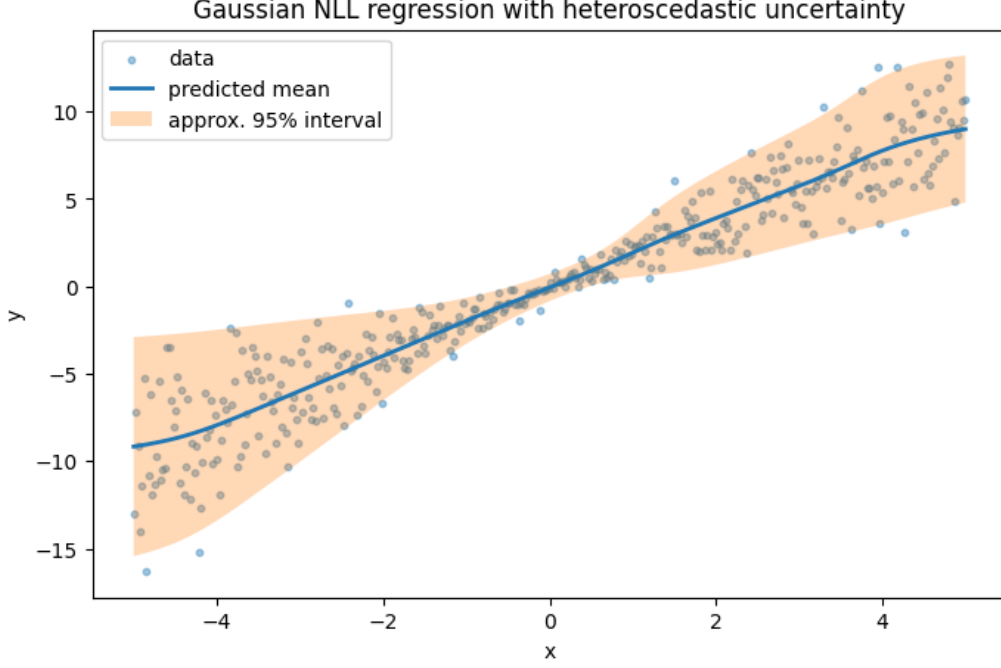


Figure 2: Heteroscedastic regression using Gaussian NLL. The predicted variance correctly increases in regions with higher noise.

These results demonstrate that Gaussian NLL produces calibrated input-dependent uncertainty estimates—an essential component for reliable cascade routing. Low-confidence predictions can thus be automatically forwarded to the more sophisticated model.

2.5 Quantile Regression (Optional Extension)

Quantile regression offers an alternative way to quantify uncertainty without assuming Gaussian noise. Instead of predicting mean and variance, the model predicts conditional quantiles:

$$q_\tau(x) \quad \text{such that} \quad P(Y \leq q_\tau(x) \mid x) = \tau.$$

Typical choices include:

$$\tau = 0.1, 0.5, 0.9,$$

forming a predictive interval:

$$[q_{0.1}(x), q_{0.9}(x)].$$

This interval can be used as a confidence measure:

- narrow interval $\Rightarrow$ high confidence (accept small-model prediction),
- wide interval $\Rightarrow$ low confidence (escalate to large model).

Quantile regression is particularly useful for asymmetric or heavy-tailed noise distributions where Gaussian assumptions may fail.

3 Models on the Income Dataset Question 3

In this section we train two probabilistic regression models on the Income dataset [2]: a small linear model and a larger multilayer perceptron (MLP), both outputting a predictive mean and an input-dependent variance. The objective is to compare their predictive accuracy and uncertainty, and later use them as candidates for the cascade system.

3.1 Data Loading and Preprocessing

We use the ACS 2018 1-year person-level survey provided by the `folktables` library, covering all 50 US states. From this dataset we extract the features defined in `ACSIncome.features` (10 demographic and socio-economic attributes) and the personal income column `PINCP` as target:

$$X \in \mathbb{R}^{N \times 10}, \quad y_{\text{raw}} \in \mathbb{R}^N.$$

The preprocessing steps are:

1. Remove rows with missing values in either features or target.
2. Clamp negative incomes to zero:

$$y_{\text{raw}} = \max(y_{\text{raw}}, 0).$$

3. Apply a log-transform to stabilize the distribution and reduce skewness:

$$y = \log(1 + y_{\text{raw}}).$$

4. Split the data into train/validation/test subsets with an 80/10/10 ratio.
5. Standardize the input features using statistics computed on the training set.

After filtering NaNs, we obtain:

$$N_{\text{total}} = 1,669,023 \text{ samples},$$

with

$$N_{\text{train}} = 1,335,218, \quad N_{\text{val}} = 166,902, \quad N_{\text{test}} = 166,903.$$

3.2 Small Model: Linear Regression with Confidence

The small model is a linear regressor for both the mean and the log-variance:

$$\mu(x) = w_{\mu}^{\top} x + b_{\mu}, \quad \log \sigma^2(x) = w_{\sigma}^{\top} x + b_{\sigma}.$$

In PyTorch, this corresponds to two linear layers without hidden units. The model is trained with the Gaussian NLL loss described in Section 2, using the Adam optimizer with learning rate 10^{-3} for 15 epochs and mini-batches of size 4096.

During training, the validation NLL steadily decreases from 27.43 to 2.75, indicating that the model is able to fit a coarse pattern in the data. However, its predictive error on the test set remains high (see Table 2), which is expected given the strong non-linearity of income as a function of the demographic attributes.

3.3 Large Model: MLP with Confidence

The large model is a feed-forward neural network with three hidden layers followed by two output heads:

$$h_1 = \phi(W_1 x + b_1), \quad h_2 = \phi(W_2 h_1 + b_2), \quad h_3 = \phi(W_3 h_2 + b_3),$$

where ϕ is the ReLU activation function. From the final hidden representation h_3 we obtain:

$$\mu(x) = w_{\mu}^{\top} h_3 + b_{\mu}, \quad \log \sigma^2(x) = w_{\sigma}^{\top} h_3 + b_{\sigma}.$$

In the experiments, we use hidden layer sizes (256, 128, 64). The training procedure is identical to that of the small model (Gaussian NLL, Adam with learning rate 10^{-3} , 15 epochs, batch size 4096). The validation NLL quickly converges from 0.37 to approximately 0.22, much lower than the linear model, suggesting a substantially better fit.

3.4 Test Performance of Small and Large Models

We evaluate both models on the held-out test set using the mean squared error (MSE) on log-income, considering only the predicted mean $\mu(x)$, and compute the average predicted standard deviation $\sigma(x)$ as a global measure of uncertainty.

Model	Test MSE (log-income)	Mean predicted σ
Small linear model	82.1782	4.8096
Large MLP model	0.6983	0.7784

Table 2: Test performance of the small and large probabilistic models on the Income dataset. The small linear model exhibits a very large error and compensates with extremely high predicted uncertainty. The large MLP achieves much lower error with moderate uncertainty, making it a suitable “accurate but expensive” component for the cascade system.

The results show a clear separation of roles: the small linear model is fast and highly uncertain, whereas the large MLP provides accurate predictions with better-calibrated uncertainty. In the subsequent sections, we will use these two models as the *fast* and *slow* components of the cascade system.

4 Question 4: Effect of Model Size on Accuracy and Inference Time

A central design choice in cascade systems is the selection of model sizes for the fast and slow stages. To understand how model capacity affects accuracy and computational cost, we evaluate a family of multilayer perceptrons (MLPs) of increasing width on a subsample of the Income dataset. This analysis quantifies the accuracy–latency trade-off that ultimately guides the choice of models for the cascade.

4.1 Experimental Setup

For efficient experimentation, we draw a random subset of

$$300\,000$$

training examples from the full training split. We define a flexible MLP architecture parameterised by a sequence of hidden-layer widths:

$$\text{MLP}(h_1, \dots, h_L),$$

where h_ℓ is the width of layer ℓ . All networks share the same input dimension $d_{\text{in}} = 10$, use ReLU activations, and terminate with two linear output heads predicting:

$$\mu(x), \quad \log \sigma^2(x),$$

as required for the Gaussian NLL loss introduced in Section 2.

We evaluate five configurations:

$$\text{tiny} = (), \quad \text{small} = (32), \quad \text{medium} = (128, 64), \quad \text{large} = (256, 128, 64), \quad \text{xlarge} = (512, 256, 128, 64)$$

Each model is trained for 3 epochs using the Gaussian NLL objective and the Adam optimiser (learning rate 10^{-3}). To assess inference cost, we measure the average time taken to process the entire test set and report mean latency per sample (ms/sample).

4.2 Results

Table 3 summarises the results. The parameter count is computed as:

$$\text{params} = \sum_{\ell=1}^L (d_{\ell-1}d_{\ell} + d_{\ell}) + 2(d_L + 1),$$

where $d_0 = d_{\text{in}}$ and d_L is the final hidden dimension. The last term accounts for the two output heads.

Model	# Params	Test MSE	Mean σ	Time (ms / sample)
tiny	22	108.7744	0.9736	0.00497
small	418	71.2010	2.9243	0.00514
medium	9 794	2.4757	1.5692	0.00537
large	44 098	1.2792	1.1118	0.00681
xlarge	178 242	0.9589	0.9762	0.00649

Table 3: Effect of model size on predictive performance and inference time. Increasing model capacity sharply improves accuracy while producing only modest increases in latency.

Figure 3 provides a visual comparison. Test MSE decreases steeply as the network width grows, whereas inference time increases only marginally.

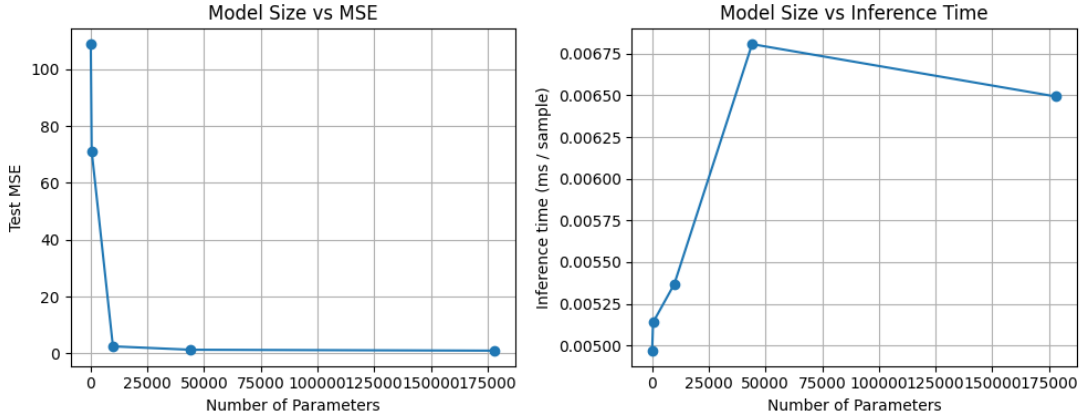


Figure 3: Model size vs. test MSE (left) and inference time per sample (right). Accuracy improves rapidly from the tiny to medium range, with only small increases in inference cost.

4.3 Selecting Models for the Cascade

The results reveal a consistent trend:

- The *tiny* and *small* models are extremely fast but far too inaccurate to serve as standalone regressors.
- Increasing to the *medium* and *large* configurations yields dramatic gains in accuracy with only slight increases in latency.
- The *xlarge* model achieves the best MSE but offers only marginal improvement over the *large* model while substantially increasing parameter count.

Based on these observations, we adopt the following for our cascade:

- a *fast* small model (the linear model from Section 3) as the first stage, due to its minimal latency and modest memory footprint;
- a *large* MLP with hidden layers (256, 128, 64) as the accurate second stage.

Overall, this analysis shows that the chosen large model lies in a favourable region of the accuracy–latency trade-off curve, providing substantial accuracy improvements for a relatively small computational cost increase—making it well suited as the “slow but accurate” component of a cascade.

5 Confidence-Based Cascade System: Question 5

We now combine the small and large models into a confidence-based cascade. The central idea is to use the *uncertainty* predicted by the small model to decide whether its output should be trusted or whether the input should instead be forwarded to the large model. This mechanism enables the cascade to allocate computational resources dynamically, processing easy inputs with a fast model and reserving the large model for ambiguous cases.

5.1 Decision Rule Based on Predicted Uncertainty

For each input x , the small model produces a mean prediction $\mu_s(x)$ and a standard deviation $\sigma_s(x)$, the latter serving as an estimate of predictive uncertainty. The large model outputs a more accurate mean prediction $\mu_\ell(x)$ (we do not use its variance here). Given a confidence threshold $T > 0$, the cascade prediction is defined as:

$$\hat{y}_T(x) = \begin{cases} \mu_s(x), & \text{if } \sigma_s(x) < T, \\ \mu_\ell(x), & \text{otherwise.} \end{cases}$$

The threshold T governs the routing behaviour:

- **Small T :** the small model is rarely trusted; most samples are forwarded to the large model.
- **Large T :** the small model is frequently trusted; only highly uncertain samples are sent to the large model.

For a given threshold T , we evaluate three quantities:

- the cascade mean squared error

$$\text{MSE}_{\text{cascade}}(T) = \frac{1}{N} \sum_{i=1}^N (\hat{y}_T(x_i) - y_i)^2,$$

- the fraction of inputs handled by the small model

$$f_{\text{small}}(T) = \frac{1}{N} \sum_{i=1}^N \mathbf{1}\{\sigma_s(x_i) < T\},$$

- the effective inference time per sample

$$t_{\text{eff}}(T) \approx f_{\text{small}}(T) t_{\text{small}} + (1 - f_{\text{small}}(T)) t_{\text{large}},$$

where t_{small} and t_{large} are obtained from Section 4.

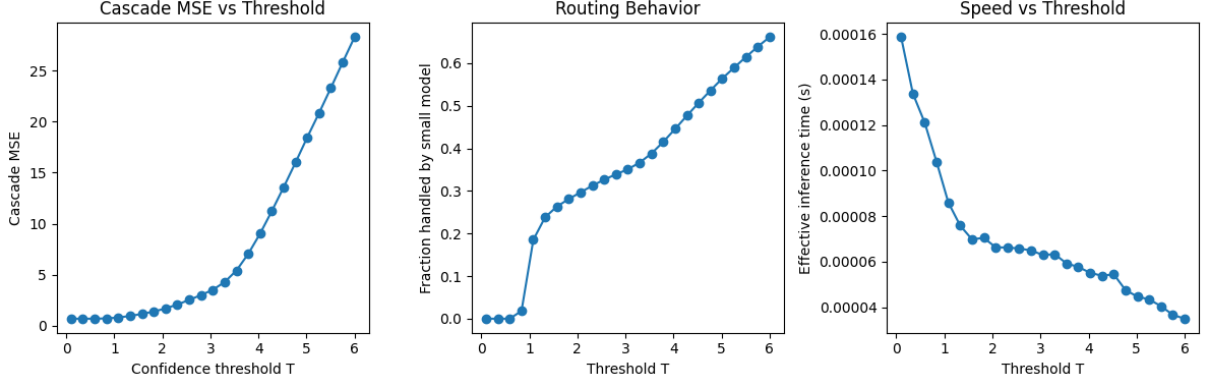


Figure 4: Behaviour of the confidence-based cascade as a function of the threshold T . Left: cascade MSE. Middle: fraction of samples processed by the small model. Right: effective inference time per sample.

5.2 Threshold Sweep and Cascade Behaviour

We evaluate the cascade for thresholds $T \in [0.1, 6.0]$. The results are visualised in Figure 4, which displays the cascade MSE, the fraction of samples handled by the small model, and the resulting effective inference time.

The left panel shows that for very small thresholds ($T \approx 0$), the cascade behaves almost identically to the large model, yielding low error but high computational cost. As T increases, more samples are routed to the small model, and the cascade MSE initially remains stable before increasing sharply once difficult samples cease to be forwarded to the large model.

The middle panel confirms this routing behaviour: the proportion of inputs treated by the small model increases smoothly from nearly 0 to over 60%.

The right panel displays the corresponding effective inference time. As T increases and more computation is performed by the faster model, latency decreases significantly. Remarkably, for intermediate thresholds the cascade achieves substantial speedups while maintaining competitive accuracy.

5.3 Selecting an Operating Point and Resource Gains

Taken together, the three curves illustrate the characteristic accuracy–efficiency trade-off of cascade systems:

- **Low thresholds** reproduce the accuracy of the large model but incur its full computational cost.
- **High thresholds** minimise computation but degrade accuracy sharply.
- **Intermediate thresholds** offer the most attractive compromise, preserving accuracy while yielding substantial efficiency gains.

In the context of edge computing, these gains translate into meaningful reductions in resource usage:

- **Computation and energy.** Fewer invocations of the large MLP reduce both compute load and power consumption on MEC nodes.
- **Bandwidth.** Only uncertain samples require offloading to a remote server, significantly reducing data transmission.

- **Latency.** Local processing of easy samples avoids network delays and lowers end-to-end response time.
- **Memory.** The small model fits comfortably on constrained devices, while the large model can remain on a remote server.

Overall, the confidence-based cascade leverages heteroscedastic uncertainty estimates to adapt computation to input difficulty, providing an effective mechanism to balance predictive accuracy against latency, bandwidth, and energy constraints in edge computing scenarios.

6 Effect of the Small-Model Loss Function on Cascade Performance: Question 6

In earlier sections, the small model was trained using a heteroscedastic Gaussian NLL loss, enabling it to output both a prediction and an input-dependent uncertainty estimate $\sigma(x)$. In this section, we examine alternative training objectives for the small model. Since these losses do not produce an uncertainty estimate directly, we also develop a secondary uncertainty-prediction network and evaluate how these changes affect the overall cascade performance.

6.1 Alternative Loss Functions for the Small Model

We consider a lightweight neural network with a single hidden layer of 32 units and a scalar output:

$$h(x) = \phi(Wx + b), \quad \mu_s(x) = w^\top h(x) + c,$$

where ϕ is a ReLU activation. Unlike the heteroscedastic model, this network outputs only a mean prediction. We train it using one of the following regression losses:

- **MSE:**

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (y_i - \mu_s(x_i))^2,$$

- **RMSE:** the square root of MSE,

- **MSLE:**

$$\mathcal{L}_{\text{MSLE}} = \frac{1}{N} \sum_{i=1}^N (\log(1 + y_i) - \log(1 + \mu_s(x_i)))^2,$$

- **MAE:**

$$\mathcal{L}_{\text{MAE}} = \frac{1}{N} \sum_{i=1}^N |y_i - \mu_s(x_i)|,$$

- **Huber loss** with parameter δ :

$$\mathcal{L}_{\text{Huber}} = \frac{1}{N} \sum_{i=1}^N \begin{cases} \frac{1}{2}(y_i - \mu_s(x_i))^2, & |y_i - \mu_s(x_i)| \leq \delta, \\ \delta|y_i - \mu_s(x_i)| - \frac{1}{2}\delta^2, & \text{otherwise.} \end{cases}$$

All models are trained on the same 300,000-sample subsample for five epochs using Adam with learning rate 10^{-3} .

6.2 Learning an Explicit Uncertainty Model

Because these small models do not predict a variance term, we learn a separate uncertainty estimator. Using the hidden representation $h(x)$ from the small model, we train an auxiliary neural network $u(h)$ to predict the absolute error:

$$e(x) = |y - \mu_s(x)|.$$

The empirical errors are normalised to $[0, 1]$ for stability, and the auxiliary model is trained to minimise the MSE between predicted and true (denormalised) errors. The output

$$\hat{e}(x) \approx \mathbb{E}[|y - \mu_s(x)| \mid x]$$

acts as a proxy for uncertainty and is used as the routing criterion in the cascade.

6.3 Cascade Based on Predicted Error

For each loss, we construct a two-stage cascade:

$$\hat{y}_T(x) = \begin{cases} \mu_s(x), & \text{if } \hat{e}(x) \leq T, \\ \mu_\ell(x), & \text{otherwise,} \end{cases}$$

where $\mu_\ell(x)$ is the large-model prediction from the heteroscedastic MLP. We vary T across a range determined by the empirical error distribution and measure:

- cascade MSE on the test set,
- the fraction of samples routed to the small model,
- effective latency combining the small and large model timing profiles.

Figure 5 shows these curves for each loss: the left column reports cascade MSE, the middle column the small-model routing fraction, and the right column the effective latency.

When the threshold is chosen solely to minimise MSE, all cascades collapse to a degenerate operating point: almost all samples are routed to the large model, yielding a cascade MSE ≈ 0.70 and offering no computational savings. This is expected since a strict MSE-optimal solution favours the most accurate model regardless of cost.

6.4 Comparison at a Balanced Operating Point

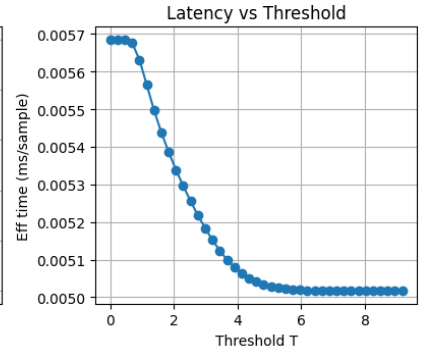
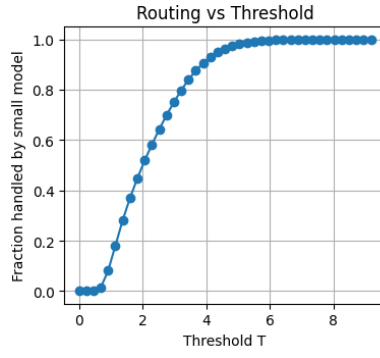
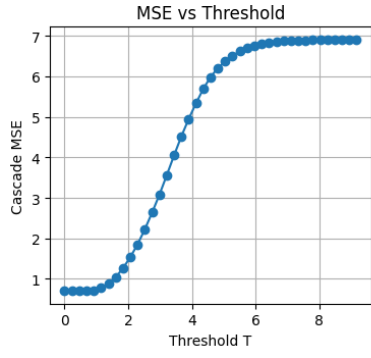
To obtain a fairer comparison, we examine a balanced operating point where approximately 25% of test inputs are handled by the small model. For each loss, we identify the threshold closest to this routing fraction and report the resulting cascade performance in Table 4.

Loss	T_{bal}	Frac. small	Cascade MSE	Eff. time (ms/sample)
MSE	1.38	0.282	0.878	5.50×10^{-3}
RMSE	1.09	0.311	1.031	5.44×10^{-3}
MSLE	4.42	0.240	3.588	6.10×10^{-3}
MAE	0.97	0.258	0.802	5.52×10^{-3}
Huber	0.92	0.312	0.779	5.80×10^{-3}

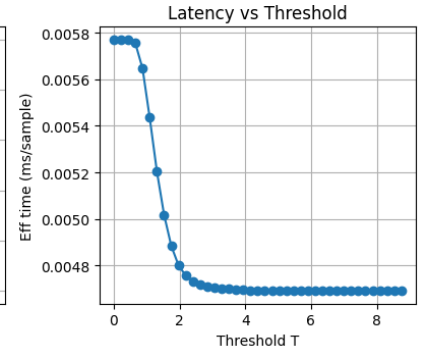
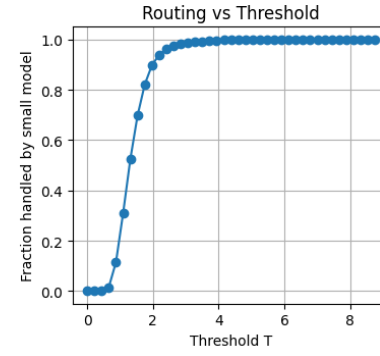
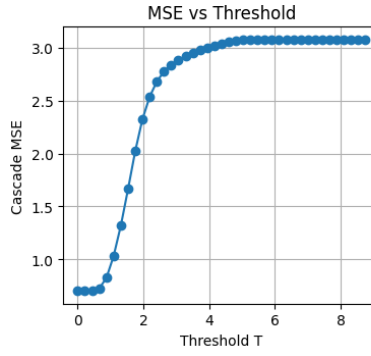
Table 4: Cascade performance for different small-model losses at an operating point where approximately 25% of samples are processed by the small model.

To complement this analysis, Figure 6 summarises the best achievable cascade MSE per loss function (left), the corresponding routing fraction (middle), and the effective latency (right). In all cases, the best MSE coincides with near-exclusive reliance on the large model.

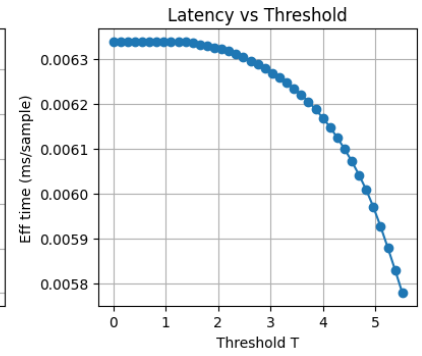
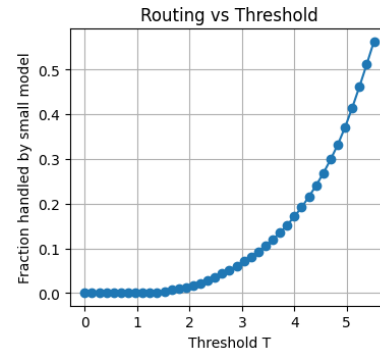
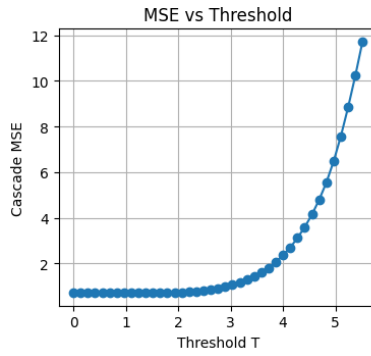
Cascade behavior for loss: MSE



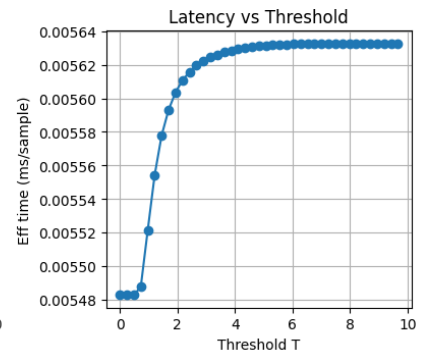
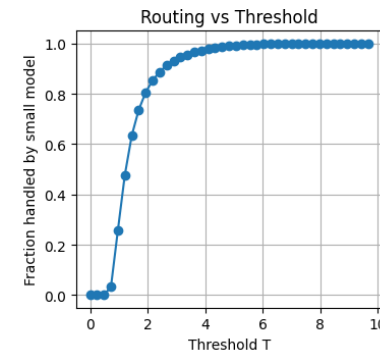
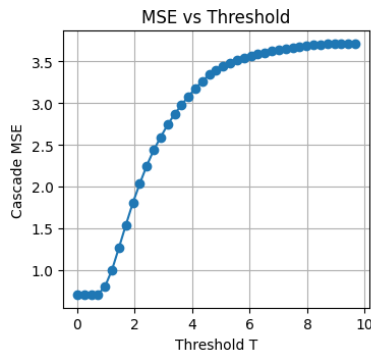
Cascade behavior for loss: RMSE



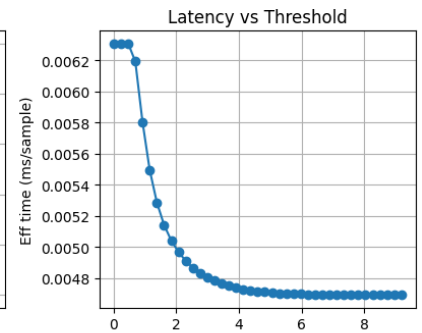
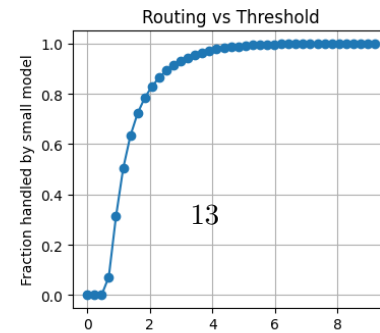
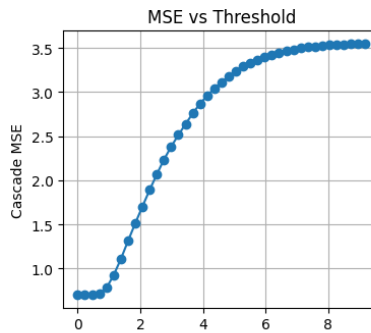
Cascade behavior for loss: MSLE



Cascade behavior for loss: MAE



Cascade behavior for loss: Huber



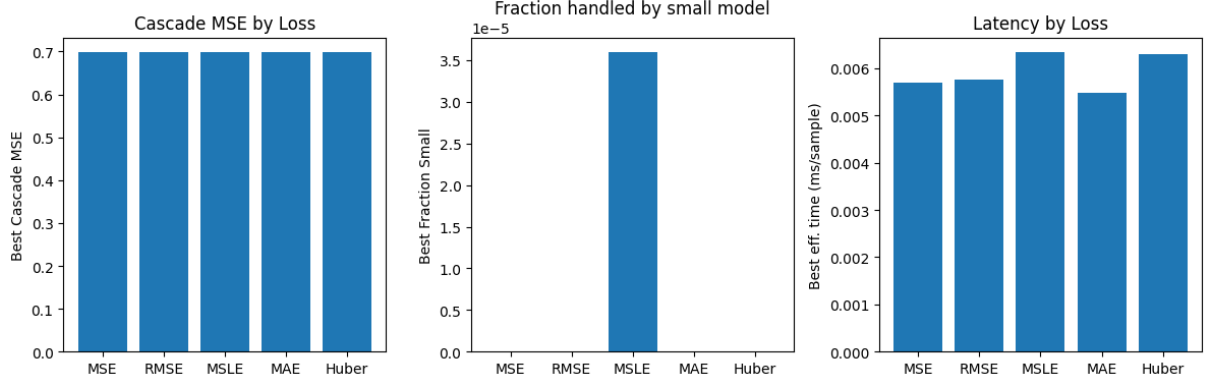


Figure 6: Best cascade performance per loss function. Left: minimum cascade MSE. Middle: fraction processed by the small model at that point. Right: corresponding effective latency. The MSE-optimal point always relies almost entirely on the large model.

6.5 Discussion

The experiments lead to several conclusions:

- When optimising purely for MSE, the cascade invariably collapses to full reliance on the large model, as this yields the lowest error but no speedup.
- At a more practically relevant operating point (approximately 25% routed through the small model), **Huber** and **MAE** losses yield the best accuracy–latency trade-offs, outperforming MSE and RMSE.
- **MSLE** performs poorly for log-income prediction, producing much higher cascade MSE and offering little advantage in latency.
- Robust losses such as MAE and Huber produce small-model embeddings that make error prediction easier for the auxiliary uncertainty network, improving routing decisions in the cascade.

Overall, the results suggest that when the small model cannot directly estimate uncertainty, pairing it with robust losses and a secondary uncertainty predictor provides a more effective basis for confidence-based cascades. In particular, Huber and MAE strike a favourable balance between robustness, quality of error prediction, and downstream cascade performance.

7 Additional Regression Task: California Housing — Question 7

To evaluate the generality of our cascade methodology, we replicate our experiments on a different regression task: predicting median house prices using the California Housing dataset. This additional benchmark allows us to assess whether the accuracy–latency trade-offs observed on the Income dataset persist across datasets with different dimensionality and noise characteristics.

7.1 Dataset and Preprocessing

We use the `fetch_california_housing` dataset from `scikit-learn`, which contains $N = 20,640$ samples and 8 numerical features per instance (e.g., median income, average rooms, population). The target variable represents median house value (in units of \$100,000). To reduce skewness and stabilise variance, we apply a logarithmic transformation:

$$y = \log(1 + y_{\text{raw}}).$$

The data are split into training, validation, and test subsets according to:

$$64\% \text{ train, } 16\% \text{ validation, } 20\% \text{ test.}$$

All input features are standardised using statistics computed on the training set.

7.2 Heteroscedastic Small and Large Models

We train two heteroscedastic MLPs analogous to those used in the Income dataset experiments:

- **Small model:** one hidden layer with 32 units,

$$\text{HeteroMLP}(32),$$

- **Large model:** three hidden layers with (256, 128, 64) units,

$$\text{HeteroMLP}(256, 128, 64).$$

Both networks output a mean $\mu(x)$ and a log-variance $\log \sigma^2(x)$ and are trained with the Gaussian NLL loss:

$$\mathcal{L}_{\text{NLL}} = \frac{1}{2} \left(\log \sigma^2(x) + \frac{(y - \mu(x))^2}{\sigma^2(x)} + \log 2\pi \right).$$

The resulting test performance is:

$$\text{Small model: } \text{MSE} = 0.3412, \quad \bar{\sigma} = 0.8950,$$

$$\text{Large model: } \text{MSE} = 0.0350, \quad \bar{\sigma} = 0.1826.$$

As in the Income experiments, the large model is substantially more accurate and exhibits lower predictive uncertainty, confirming its suitability as the second stage of the cascade.

7.3 Cascade Behaviour on California Housing

We construct a two-stage cascade using the same decision rule as in Section 5: the prediction from the small model is accepted if its predicted uncertainty satisfies $\sigma_s(x) \leq T$, otherwise the input is forwarded to the large model. We sweep T from 0.1 to 3.0 and compute:

- the cascade MSE,
- the fraction of samples handled by the small model,
- the effective latency per sample, combining the small and large model inference times.

The results are presented in Figure 7.

For low thresholds ($T \leq 0.6$), almost all samples are routed to the large model, yielding a cascade MSE close to that of the large model (≈ 0.04). As T increases toward 1.0, a sharp transition occurs: the small-model routing fraction rises rapidly toward 1, and the effective latency converges to that of the small model. In this regime, the cascade MSE approaches the small model’s error (≈ 0.34).

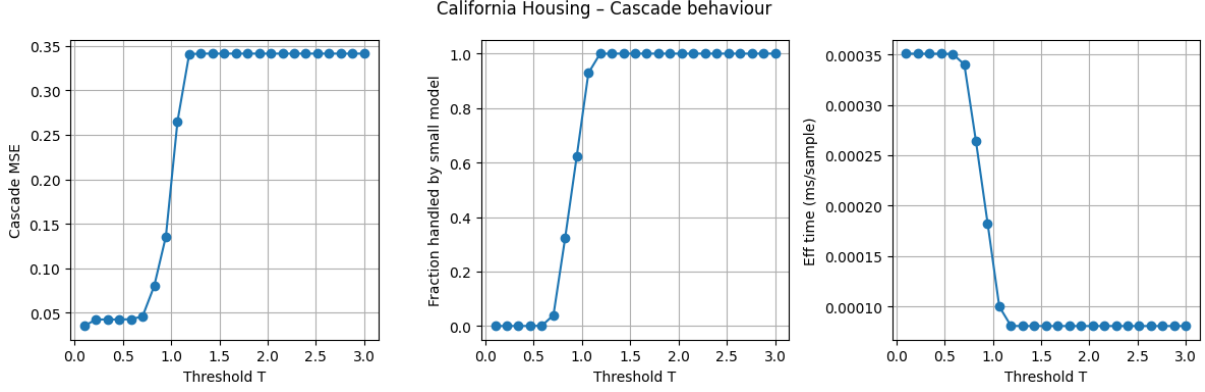


Figure 7: Cascade behaviour on the California Housing dataset. Left: cascade MSE as a function of threshold T . Middle: fraction of samples processed by the small model. Right: effective latency per sample.

7.4 Discussion

Compared to the Income dataset, the California Housing task is notably simpler and well captured by a low-dimensional representation. Consequently, even the small model achieves reasonable accuracy. This leads to a clear *phase transition* in the cascade:

- For conservative thresholds, the cascade effectively behaves like the large model, offering high accuracy at the cost of higher latency.
- For permissive thresholds, the cascade behaves like the small model, trading accuracy for significantly lower latency and computational cost.

These results reinforce the generality of the confidence-based cascade mechanism. Although the precise shape of the trade-off curve depends on how much additional accuracy the large model provides over the small one, the qualitative behaviour remains consistent across datasets: adjusting the threshold T provides a smooth and interpretable trade-off between resource usage and predictive performance.

8 Alternative Training Procedure: Specialized Large Model — Question 8

In the previous experiments, the large model was trained on the full ACS Income dataset. In this section, we investigate an alternative strategy inspired by the idea of *specializing* the large model on the most challenging inputs. The motivation is that a large model tailored to the “hard” regions of the input space might improve cascade performance, particularly at thresholds where the second stage is invoked frequently.

8.1 Hard-Sample Selection

We begin by training a small heteroscedastic MLP with one hidden layer of 32 units on the complete ACS Income training set using Gaussian NLL. This model achieves a test MSE of

$$\text{small model MSE} = 0.7675.$$

We then compute the absolute prediction error on each training example:

$$e(x_i) = |y_i - \mu_s(x_i)|.$$

Hard samples are defined as those whose error exceeds the 70th percentile of the error distribution:

$$e(x_i) \geq \tau_{0.7},$$

yielding approximately 30% of the data:

$$\text{fraction hard} \approx 0.30, \quad |\mathcal{D}_{\text{hard}}| = 400,566.$$

8.2 Baseline vs. Specialized Large Model

Using the same architecture as in earlier sections, we train two large heteroscedastic MLPs with hidden layer sizes (256, 128, 64):

- **Baseline large model:** trained on the full dataset $\mathcal{D}_{\text{train}}$,
- **Specialized large model:** trained only on the hard subset $\mathcal{D}_{\text{hard}}$.

Both models train for 15 epochs with Gaussian NLL and are evaluated on the identical full test set. The resulting test MSEs are:

$$\text{Baseline large: } \text{MSE} = 0.7019, \quad \text{Specialized large: } \text{MSE} = 0.7710.$$

Training exclusively on hard samples thus worsens overall generalisation. We also measure per-sample inference time:

$$t_{\text{small}} \approx 0.0000 \text{ ms}, \quad t_{\text{large}} \approx 0.0006 \text{ ms}, \quad t_{\text{large-hard}} \approx 0.0005 \text{ ms}.$$

The specialized large model is slightly faster, but the difference is minimal.

8.3 Cascade Comparison

We construct two cascades using the same small model and the decision rule introduced in Section 5:

- **Baseline cascade:** small model + baseline large model,
- **Specialized cascade:** small model + specialized large model.

For thresholds $T \in [0.1, 3.0]$, we evaluate:

- the cascade MSE,
- the fraction of samples routed to the small model,
- the effective latency combining t_{small} and the corresponding large-model runtime.

Figure 8 illustrates the results. Since the routing rule depends only on the small model, both cascades process the same fraction of samples for each threshold T ; differences arise only from the large-model predictions and inference times.

For low thresholds—where most inputs are forwarded to the second stage—the baseline cascade clearly achieves lower MSE, reflecting the superior global performance of the baseline large model. As T increases, the influence of the large model diminishes and both cascades converge toward the small model’s error.

In terms of latency, the specialized cascade is marginally faster due to the slightly lower per-sample time of the hard-only model, but the gain is negligible and consistently offset by worse accuracy.

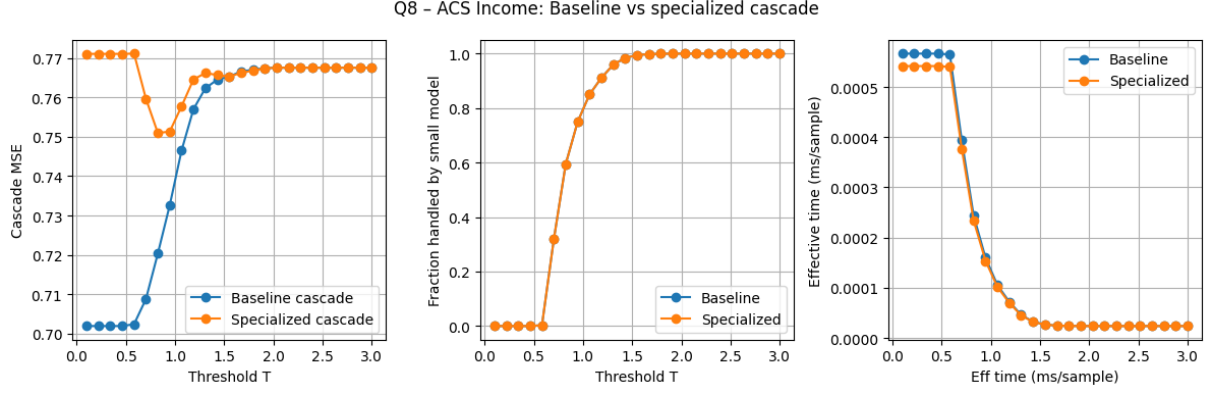


Figure 8: Question 8: comparison of baseline and specialized cascades on the ACS Income dataset. Left: cascade MSE. Middle: fraction routed to the small model (identical across cascades). Right: effective latency per sample.

8.4 Conclusion on the Specialized Training Procedure

This experiment demonstrates that restricting the large model to hard samples does not improve cascade performance. Specifically, the specialized large model:

- exhibits worse test MSE than the baseline model,
- yields a cascade with higher error across most thresholds,
- offers only minimal latency benefits.

These findings suggest that the large model benefits from exposure to the full data distribution, even if it is later applied primarily to challenging cases in the cascade. More promising alternatives may involve *sample re-weighting*, rather than discarding data, or hybrid approaches combining hard-sample emphasis with regularisation or multi-task objectives to preserve global generalisation.

References

- [1] P. Viola and M. Jones, “Rapid object detection using a boosted cascade of simple features,” in *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, vol. 1, pp. I–I, 2001.
- [2] D. Ding, N. J. Hou, M. T. Chao, and H. Hwang, “Income Prediction Using U.S. Census Microdata,” 2024. (Dataset reference used in the folkttables package.)